VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY

INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Data Science and Data Visualization Lab 03

Lab 3 — D3.js: Scales, Axes, and Visual Encoding

Student name:

Pham Nhu Nguyen Trung - ITCSIU24091

Nguyen Quynh Anh - ITCSIU24005

This is all my own work. I did not copy the code from any other source.

7.1 Domain and range

Consider:

```
const xScale = d3.scaleLinear()
domain([0, 1000])
range([40, 560]);
```

1. What is the input domain? **[0, 1000]**
2. What is the output range? **[40, 560]**
3. What output should xScale(0) produce? **40**
4. What output should xScale(1000) produce? **560**
5. Approximately what output should xScale(500) produce? **300**
6. If the population maximum changes from 1,000 to 2,000, why might a dynamic domain be better than keeping [0, 1000]?

With the domain [0,1000] the value 2000 would be out of range [40,560]. Points would spill out the chart area and be clipped or misplaced. A dynamic domain (e.g, .domain([0, d3.max(dataset)]), adapts automatically to the data so the charts keep working without manual code changes

7.2 Why invert the y-scale?

Consider:

```
const yScale = d3.scaleLinear()
domain([0, 100])
range([260, 20]);
```

1. Why does the range go from 260 to 20 rather than 20 to 260? **Because in the svg's y-coordinate gradually increases from top to the bottom (y=0 is the top). So using range [260,20] → y=260 is the bottom range and y=20 is the top range**

2. Where will a value of 100 appear: near the top or near the bottom? **yScale(100) = 20 → Near top**

3. What misleading result could occur if the y-scale is not inverted?

If the y-scale is not inverted, the low value would be in the top and the high value would be in the bottom → it can lead to hard to understand for viewers - rising trend looks like a decline

7.3 Choose the correct visual encoding

Match each data field to its intended visual property:

Data field Visual property

Population x-position

GRDP per person y-position

Province area Circle area

Population density Discrete color

Then answer:

1. Why is circle area a more appropriate description than “circle radius is proportional to area”? **Because the area = pi*r^2 so the radius should be proportional to the square root of the value**

2. Why should the color scale be discrete if the task asks for categories? **When the task asks for categories, if the colors are not discrete, it would produce many similar shades → hard to discriminate between groups. Thus, the discrete scale puts value into the clear groups.**

3. What must you inspect in the CSV before defining the scales?

Inspect the column names and data types (CSV values load as strings, so convert them to numbers), the minimum and maximum values for the domain, missing or invalid values, zeros and negatives, outliers and skew (which may call for a log or sqrt scale), the units of each field, and the number format and text encoding.

7.4 Predict before running

If the CSV contains 63 valid province records and the code uses one circle per record, how many circles should appear? **63 circles should appear because one circle per record**

What might cause fewer circles to appear?

It is caused by 2 cases:

1. Not created records drop while loading (return null data), a wrong data join, duplicated keys in key function, …

2. Created but not visible: value parsed as NaN(wrong column name, commas in numbers, empty cells), a domain or range pushes circles outside range of SVG, a radius of zero or near zero, circles overlapping or hidden behind others, …

Keep your answers. The instructor may ask you to explain one during the Group Check.

8. Build

8.1 Create the page and load the data

International University · Data Science and Data Visualization

Lab 3

Lab 3 — D3.js: Scales, Axes, and Visual Encoding

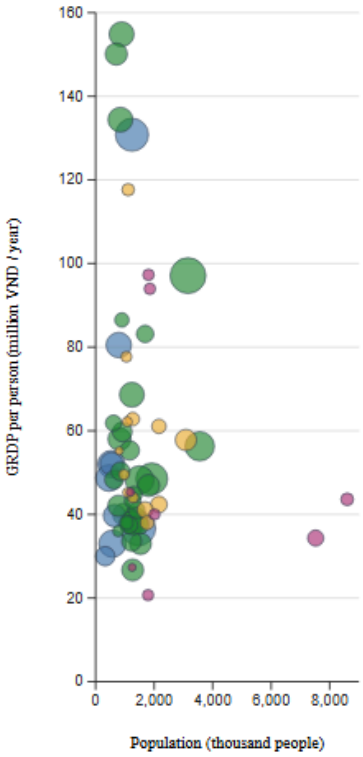
Pham Nhu Nguyen Trung — ITCSIU24091
Nguyen Quynh Anh — ITCSIU24005

This is all my own work. I did not copy the code from any other source.

Population and prosperity across Viet Nam

Each circle represents one province. Circle area represents land area.

63 provinces plotted · 0 invalid rows excluded · 300 × 600 px canvas



Population density

People per km²

- < 100
- 100 to < 500
- 500 to < 1,000
- ≥ 1,000

Fixed descriptive bands: 100, 500, and 1,000 people/km². These are chart design thresholds, not official classifications. A boundary value belongs to the higher band.

Reading the chart

Both axes start at zero because population and income have meaningful zero baselines. Upper limits come from the loaded data.

Radius is proportional to the square root of land area, so the visible circle area is proportional to province area.

Transparent fills and thin outlines help reveal overlap. Every valid record is retained; some circles can still obscure others on this narrow canvas.

Source: vn-provinces-data.csv, supplied by the lab. Population: thousand people; GRDP: million VND/person/year; land area: km²; density: people/km². Values are displayed as supplied, without correcting possible source anomalies.

8.2 Build the scatterplot

- ☑ Load vn-provinces-data.csv using D3.

☑ Use an SVG initially sized 300 pixels wide × 600 pixels high, as specified in the original handout.

```
      <svg      id="scatterplot"      width="450"      height="900"      role="img"
aria-labelledby="svg-title svg-description">
```

☑ Draw one circle for each valid province record.

```
plot
        .append("g")
        .attr("class", "provinces")
        .selectAll("circle")
        .data([...data].sort((a, b) => b.area - a.area))
        .join("circle")
        .attr("cx", (d) => xScale(d.population))
        .attr("cy", (d) => yScale(d.grdp))
        .attr("r", (d) => areaScale(d.area))
        .attr("fill", (d) => densityColor(d.density))
        .attr("fill-opacity", 0.65)
        .attr("stroke", "#334155")
        .attr("stroke-width", 0.65)
        .append("title")
                                    .text((d)     =>     `${d.province}\nPopulation:
${d.population.toLocaleString("en-US")} thousand people\nGRDP per person: ${d.grdp}
million     VND/year\nArea:     ${d.area.toLocaleString("en-US")}     km²\nDensity:
${d.density.toLocaleString("en-US")} people/km²`)
```

☑ Map population to the x-position with a linear scale.

```
const xScale = d3
        .scaleLinear()
        .domain([0, d3.max(data, (d) => d.population) || 1])
        .nice()
        .range([maxRadius, innerWidth - maxRadius])
```

☑ Map GRDP per person to the y-position with a linear scale; larger values must appear higher on the chart.

```
const yScale = d3
        .scaleLinear()
        .domain([0, d3.max(data, (d) => d.grdp) || 1])
        .nice()
        .range([innerHeight - maxRadius, maxRadius])
```

☑ Derive scale domains from the actual data rather than hard-coding observed minimum and maximum values.

```
// Population domain
.domain([0, d3.max(data, d => d.population) || 1])

// GRDP domain
.domain([0, d3.max(data, d => d.grdp) || 1])

// Province area domain
.domain([0, d3.max(data, d => d.area)])
```

☑ Leave enough inner space for axes and labels (use margins/padding).

```
const yScale = d3
        .scaleLinear()
        .domain([0, d3.max(data, (d) => d.grdp) || 1])
        .nice()
        .range([innerHeight - maxRadius, maxRadius])
```

☑ Draw a bottom x-axis and a left y-axis.

```
plot
        .append("g")
        .attr("class", "axis x-axis")
        .attr("transform", `translate(0,${yScale(0)})`)
        .call(
            d3
                .axisBottom(xScale)
                .ticks(4)
                .tickFormat(d3.format(",.0f"))
                .tickSizeOuter(0),
        )
plot
        .append("g")
        .attr("class", "axis y-axis")
        .attr("transform", `translate(${xScale(0)},0)`)
        .call(d3.axisLeft(yScale).ticks(6).tickSizeOuter(0))
```

☑ Add readable axis titles and tick labels with units where appropriate.

```
plot
        .append("text")
        .attr("class", "axis-title")
```

```
        .attr("text-anchor", "middle")
        .attr("x", innerWidth / 2)
        .attr("y", innerHeight + 35)
        .text("Population (thousand people)")
    plot
        .append("text")
        .attr("class", "axis-title")
        .attr("text-anchor", "middle")
        .attr("transform", "rotate(-90)")
        .attr("x", -innerHeight / 2)
        .attr("y", -44)
        .text("GRDP per person (million VND / year)")
```

☑ Map province area to circle area using a square-root scale or an equivalent mathematically justified mapping.

```
const areaScale = (reproduceAreaBug ? d3.scaleLinear() : d3.scaleSqrt())
        .domain([0, d3.max(data, (d) => d.area)])
        .range([0, maxRadius])
```

```
plot
        .append("g")
        .attr("class", "provinces")
        .selectAll("circle")
        .data([...data].sort((a, b) => b.area - a.area))
        .join("circle")
        .attr("cx", (d) => xScale(d.population))
        .attr("cy", (d) => yScale(d.grdp))
        .attr("r", (d) => areaScale(d.area))
        .attr("fill", (d) => densityColor(d.density))
        .attr("fill-opacity", 0.65)
        .attr("stroke", "#334155")
        .attr("stroke-width", 0.65)
        .append("title")
        .text((d) => `${d.province}\nPopulation:
${d.population.toLocaleString("en-US")} thousand people\nGRDP per person: ${d.grdp}
million VND/year\nArea: ${d.area.toLocaleString("en-US")} km²\nDensity:
${d.density.toLocaleString("en-US")} people/km²`)
```

☑ Map population density to discrete colors using a documented set of categories or thresholds.

```
const densityColor = d3.scaleThreshold().domain(thresholds).range(colors)
plot
        .append("g")
        .attr("class", "provinces")
        .selectAll("circle")
        .data([...data].sort((a, b) => b.area - a.area))
        .join("circle")
        .attr("cx", (d) => xScale(d.population))
        .attr("cy", (d) => yScale(d.grdp))
        .attr("r", (d) => areaScale(d.area))
        .attr("fill", (d) => densityColor(d.density))
        .attr("fill-opacity", 0.65)
        .attr("stroke", "#334155")
        .attr("stroke-width", 0.65)
        .append("title")
                                    .text((d)     =>     `${d.province}\nPopulation:
${d.population.toLocaleString("en-US")} thousand people\nGRDP per person: ${d.grdp}
million     VND/year\nArea:     ${d.area.toLocaleString("en-US")}     km²\nDensity:
${d.density.toLocaleString("en-US")} people/km²`)
```

☑ Include a legend that explains the density colors.

```
const legend = d3
        .select("#density-legend")
        .selectAll("li")
        .data(labels)
        .join("li")
    legend.selectAll("*").remove()
    legend
        .append("span")
        .attr("class", "swatch")
        .style("background-color", (d) => d.color)
        .style("opacity", 0.65)
    legend.append("span").text((d) => d.label)
```

☑ Include a chart title and a short note describing the source and units

```
<p class="source">Source: <a
href="https://tungth.github.io/data/vn-provinces-data.csv">vn-provinces-data.csv</a
>, supplied by the lab. Population: thousand people; GRDP: million VND/person/year;
land area: km²; density: people/km². Values are displayed as supplied, without
correcting possible source anomalies.</p>
```

9. Verify

9.1. Verification checklist

- ☑ The number of circles equals the number of valid records loaded.
- ☑ The browser console shows no uncaught errors.
- ☑ Population and GRDP fields are numeric, not strings.
- ☑ The smallest and largest values fall within the plotting area.
- ☑ The y-axis increases in value from bottom to top.
- ☑ The axes align with the chart's plotting area.
- ☑ Every density category in the legend matches the circle colors.
- ☑ A larger province area produces a larger circle area, not merely a proportionally larger radius.
- ☑ Axis titles and units are understandable without reading the code

Step 1: State hypotheses before running the checks

- If every valid CSV row becomes a mark, circle count will equal valid record count.
- If conversion is correct, all four numeric measures will be finite numbers, including decimal-comma GRDP values.
- If dynamic scales and radius padding are correct, all circle edges will remain inside the plotting area.
- If the y range is reversed, larger GRDP values will have smaller SVG y coordinates.
- If size encodes land area correctly, the ratio of rendered radius squared will equal the ratio of land areas.
- If the legend uses the chart's threshold rule, every swatch and circle will use the expected color; an exact boundary will enter the higher band.

Step 2: Add repeatable checks

Added verifyChart() in script.js. It inspects actual circle attributes (cx, cy, r, and fill), SVG dimensions, axis transforms, legend swatches, numeric values, and the scale

domains. Results appear in the page and console. It reports failures without silently removing records.

Step 3: Run the normal source chart and record evidence

Requirement/test	Observed evidence	Result
Circle count equals valid rows	63 loaded, 63 valid, 63 SVG circles; 0 excluded	PASS
No uncaught errors	Browser error log returned no entries	PASS
Numeric conversion	Population, GRDP, area and density are finite JavaScript numbers	PASS
Minimum and maximum fit	Every circle, including its radius, fits within the 368 × 814 inner area	PASS
Conventional y direction	y(0) = 801; y(154.84) = 38.413, so the larger value is higher	PASS
Axes align with plotting area	Bottom axis translated to y=801; left axis to x=13; both meet at zero	PASS
Dynamic domains	Population extent 327.9–8598.7 → domain [0, 9000]; GRDP extent 20.7–154.84 → domain [0, 160] after .nice()	PASS
Density legend matches circles	All four actual swatch colors and all 63 circle fills match the shared mapping	PASS
Boundary behavior	Tested just below and exactly at 100, 500 and 1000; exact values enter the higher band	PASS
Circle area proportional to land area	Rendered radius² ratios match source land-area ratios for all marks	PASS
Readable titles and units	Visually inspected full chart: population in thousand people; GRDP in million VND/person/year; density and area units stated	PASS

The on-page source view has 11/11 automatic checks passing, including the size and tooltip checks added for Section 10. Console error inspection and visual label inspection are additional checks, not included in that total.

Section 9.2 — Test with small data

Define a test dataset using the actual field structure

The test CSV uses province, population, GRDP-VND, area, and density. GRDP values retain the decimal-comma format so the real conversion path is tested.

Province	Population (thousand)	GRDP (million VND/year)	Area (km²)	Density (people/km²)
Test Low	100	10	100	99
Test Medium	500	50	400	100
Test High	1000	100	900	500
Test Maximum	1500	150	1600	1000

These are synthetic test values, not real provinces. They cover low, medium and high positions, maximum area, and all three exact density boundaries.

Run the small-data view

Clicked Small test dataset, which loads index.html?test=1 through D3 and the same buildChart() function as the source data.

Observed results:

1. Four valid rows produced four circles.
2. The points move right and upward in low → medium → high → maximum order.
3. Domains recalculated to x=[0,1600] and y=[0,160], proving they respond to the replacement data.
4. Land areas 100, 400, 900 and 1600 produce radii 3.25, 6.5, 9.75 and 13 pixels.
5. A fourfold land-area increase (100 → 400) produces a fourfold circle-area increase: $(6.5 / 3.25)^2 = 4$.
6. The maximum area gives the maximum configured radius of 13 pixels.
7. Density 99 is blue; 100 is green; 500 is yellow; 1000 is purple. These match the legend.

8. All circle edges remain inside the adapted plotting area.

Result: 14/14 automatic checks passed.

9.3 Debugging challenge: Bug C

Step 6: Reproduce the defect deliberately

The existing source chart already used square-root sizing. To test the defect honestly, added a controlled reproduction: index.html?test=1&bug=area. This switches only the synthetic test view to d3.scaleLinear() for radius. The source view always uses d3.scaleSqrt().

1. Observed: Test Medium has four times Test Low's land area, but sixteen times its rendered circle area. The reproduction reports 12/14 checks passed, with both proportional-area checks failing.
2. Expected: Four times the land area should give four times the circle area, hence twice the radius.
3. Likely cause: Radius is linear in land area; because circle area is πr^2, this squares the intended ratio.
4. Fix: Use d3.scaleSqrt().domain([0, maxArea]).range([0, 13]). A zero origin is necessary for exact proportionality.
5. Verification: Reopened Small test dataset. Observed circle-area ratio returned from 16 to 4; 14/14 checks passed. The unchanged maximum-area radius of 13 shows that the failing test detects the relationship between sizes, rather than only checking the largest mark.

Step 7: Restore the source CSV

Clicked Source CSV. Confirmed 63 circles, 0 excluded records, 11/11 checks passed, and no browser error log entries. The submission view therefore displays actual province data.

10 - Adapt

Prediction note

Suggested prediction: “Changing width and height should update the inner dimensions, scale ranges, axes and label positions; adding tooltips should change each circle's hover text without changing its position, size or color.”

This sentence is an AI-assisted draft. The handout asks for a prediction written before using AI; this document does not claim that the students independently wrote it before this session. Add your own original prediction if you recorded one.

Step 8: Change the canvas from 300 × 600 to 450 × 900

1. Changed the width and height constants in script.js to 450 and 900.
2. Updated the initial SVG attributes in index.html to the same dimensions.
3. Updated the SVG flex width in style.css to 450 pixels, preserving the simple styling.
4. Retained margins { top: 24, right: 18, bottom: 62, left: 64 }; inner dimensions therefore change from 218 × 514 to 368 × 814.
5. Because the scales already derive their ranges from inner dimensions, x changes from [13,205] to [13,355] and y from [501,13] to [801,13].
6. The bottom axis still uses yScale(0) and the left axis still uses xScale(0). Titles and gridlines continue to use inner dimensions, so their positions update automatically.
7. Made the status text use the dimension variables instead of a fixed “300 × 600” message.

Step 9: Choose additional adaptation B — Tooltip

Expanded each circle's SVG <title> from the province name to five fields: province name, population, GRDP per person, land area, and density. Numeric fields include units. The browser displays this native tooltip when a circle is hovered; no extra styling or chart library is required.

Step 10: Verify the adaptation

- Browser inspection confirmed actual SVG attributes of 450 × 900.

- Automated geometry inspection confirmed every circle fits inside the 368 × 814 plotting area.
- Visually inspected the complete chart: circles, axes, tick labels and axis titles remain inside the SVG canvas.
- Source and small-data tests still passed, preserving the population, GRDP, land-area and density encodings.
- Inspected every circle's <title>: all contain the province name and the four labeled measures. The tooltip content was verified; the browser-controlled test did not capture a native tooltip popup on screen.

Reflection

1. **What was easiest to adapt?** Changing the two dimension constants, because scale ranges, gridlines, axes and labels already depend on inner dimensions.
2. **What depended on hard-coded assumptions?** Margins, maximum radius of 13 pixels, tick counts, density boundaries and CSS flex width. The expected 450 × 900 size in the verification check is intentional for this task. The data domain maxima are calculated dynamically.
3. **What would improve with more records or larger values?** Larger values already expand the domains. More records could increase overlap; consider highlighting/search and a larger plotting area. Long tick labels may require wider margins. The embedded CSV and external CSV must be kept in sync when updating the dataset.